



School of Informatics

Informatics Project Proposal

Unified Multi-Tenant Inference Runtime for Edge AI on Latency-Critical Shared RAN Infrastructure

UID: s2880330

Date: April 2026

Tutor: Dr. James Garforth

Supervisor: Prof. Mahesh K. Marina

I give permission for future Informatics Project Proposal classes to read an anonymous version of this document.

This page does not count towards the page limit.

Abstract

As 6G and AI-enabled Radio Access Network (AI-RAN) architectures mature, the Radio Access Network (RAN) is increasingly treated not only as communications infrastructure but also as a distributed accelerator platform for edge AI services. The unresolved systems question is not whether such infrastructure occasionally exposes transient resources, but how heterogeneous inference workloads can consume bounded, revocable execution opportunities without violating strict QoS preservation for primary radio functions. To operationalize that question, the project develops a unified multi-tenant inference runtime for shared RAN infrastructure around bounded execution windows, workload-aware segmentation, context offloading, hard memory budgeting, and preemption-safe execution. Rather than treating inference as a GPU-only task, the runtime explicitly coordinates the radio uplink for input data, the accelerator compute phase, and the downlink for results so that network bottlenecks and accelerator scheduling decisions are managed together. Methodologically, the work profiles representative workload families, derives a shared runtime-control abstraction, implements the design in a bounded prototype, and evaluates whether useful goodput can be delivered while remaining subordinate to RAN-first timing constraints.

Declaration of Own Work

I hereby declare that the work presented in this document is my own. The use of Generative AI to assist in the formatting and structuring of this proposal has been fully acknowledged.

1. Introduction

1.1. Problem

AI-RAN and Open RAN research increasingly suggest that compute deployed near the Radio Access Network (RAN) – the distributed edge infrastructure of cell sites, base stations, and associated control software that connects user devices to the core network – need not serve radio processing alone; under the right conditions, it can also host selected edge AI services [1, 2, 3]. At the same time, the literature already treats exported RAN-edge capacity as transient rather than guaranteed: capacity can appear outside periods of peak radio demand and then disappear again as radio load rises.

The central systems consequence is that inference cannot be designed as if execution will continue uninterrupted once admitted. Because the radio stack may reclaim resources at any millisecond, an inference runtime operating on RAN-edge hardware must assume that execution is inherently volatile. Any service placed on such infrastructure must therefore make progress in bounded, interruptible units rather than depend on long, continuous accelerator ownership. That requirement is especially difficult because edge inference is not a single workload class. A stateless vision inference service, for example, typically consumes one image, executes one forward pass, and returns one result. A streaming large language model (LLM) service, by contrast, generates output token by token and must preserve context across successive decoding steps. Sequence-to-sequence translation sits between those extremes: it preserves encoder-decoder state across sentence-level generation, but still exposes clearer completion units than open-ended language generation. Automatic Speech Recognition (ASR) introduces another execution regime because the system ingests streaming audio chunks and must reconcile partial hypotheses under tight end-to-end delay budgets. What counts as an execution unit, how much state must survive preemption, and where useful work is lost therefore differ substantially across workloads. In this setting, the key question is not simply whether transient resources exist, but how to use revocable compute safely without weakening strict QoS preservation for primary radio processing.

1.2. Idea

The proposal rests on a simple systems principle: if RAN-edge accelerators are transient resources, the inference stack needs an explicit contract that allows those resources to be borrowed safely and returned immediately when the radio requires them. The objective is not to redesign radio scheduling, nor to treat the RAN as a generic edge cloud. Instead, the design makes inference execution compatible with a volatile environment in which radio resources and compute resources are coupled, and in which revocation is a normal operating condition rather than an exception.

To operationalize that principle, the proposal introduces **ComputeLease**, an inference-side contract that specifies the duration of a bounded execution window, the permitted share of accelerator capacity, the memory limit, the radio-side budget or stage-pressure assumptions under which execution remains admissible, and the preemption behaviour that applies when the RAN takes resources back. The reason such a contract is necessary is that workload diversity translates directly into control complexity: a runtime must reason about the execution unit that can be scheduled or preempted, the persistent state that must be retained to resume correctly, the safest interruption boundary, and the accelerator pressure imposed by the workload while it runs. Those issues are lightweight for stateless vision inference, but they are central for streaming services whose useful work depends on preserving context across successive decoding steps. In effect, **ComputeLease** defines the conditions under which a workload can consume transient

compute resources while remaining subordinate to strict QoS preservation for the radio stack.

The hypothesis is that heterogeneous inference services can coexist safely on shared RAN infrastructure if the runtime is designed around that contract from the outset. Under this view, compatibility depends not only on raw accelerator demand, but also on whether a workload can execute in bounded windows, whether its context can be preserved efficiently through context offloading across preemption, and whether useful goodput can still be delivered under revocable execution. At the level of the introduction, the core idea is that volatile RAN-edge compute requires contract-based, preemption-aware inference control rather than unstructured opportunistic sharing.

1.3. Importance

This problem matters for three connected reasons: (i) at the infrastructure level, it addresses a live AI-RAN question now visible across the Open RAN and AI-RAN literature, namely whether operators can convert transient resources near the Distributed Unit (DU) into a safe substrate for native AI services rather than leaving that capacity idle [1, 2]; (ii) at the workload level, edge inference is not one homogeneous class, because stateless vision inference, streaming language generation, sequence-to-sequence translation, Automatic Speech Recognition (ASR), and rendering-oriented services differ in execution unit, persistent state, safe interruption boundary, and accelerator pressure [4, 5, 6, 7]; and (iii) at the systems level, a workable solution would strengthen the case for low-latency AI support on constrained endpoints while providing operators with a more credible route to infrastructure reuse.

1.4. Approach

Methodologically, the project follows a systems-oriented workflow. It begins by profiling representative workload families from the literature and identifying their execution units, persistent-state requirements, and resource sensitivities. It then maps those workload properties to serving patterns already described in the systems corpus and uses that mapping to derive a common runtime-control abstraction. The next step is to implement that abstraction in a bounded prototype and evaluate whether the resulting design can preserve strict QoS while still delivering useful goodput inside revocable execution windows. In consequence, the project remains dissertation-scale while still producing concrete design artefacts, a working prototype, and evidence-based evaluation.

1.5. Plan

To keep the work thematically aligned, the dissertation is organised around four work packages that map directly onto the argument of the proposal. WP1 establishes the workload and design baseline; WP2 defines the unified runtime-control abstraction and the shared mechanism set; WP3 implements the prototype and workload adapters; and WP4 evaluates the system while consolidating the dissertation text and preserving contingency for integration and revision. Each stage exists to answer a specific research question with an explicit artefact and evaluation target.

1.6. Success

With respect to workload compatibility, the project succeeds if it can formalise a defensible baseline for representative workload families and answer **RQ1**: which edge inference workloads are technically compatible with shared RAN infrastructure, and why? The first objective is

therefore to identify execution units, persistent state, dominant pressure points, and safe interruption boundaries for each representative family [4, 5]. The associated evaluation criterion is that the workload matrix must separate compatible from incompatible families using explicit evidence rather than intuition.

At the runtime-design stage, success requires an answer to **RQ2**: what runtime-control abstraction and shared mechanism set are needed to manage those workloads under bounded, revocable execution? The second objective is to specify the common mechanism set and the workload-specific adapters needed for a unified design. The associated evaluation criterion is that the runtime architecture must map every major control decision — admission, segmentation, context offloading, tiered resource isolation, that is, progressively stricter separation among competing workloads, memory budgeting, and revocation handling — back to a workload property established in the baseline.

For end-to-end system validation, success requires an answer to **RQ3**: can a prototype built around that abstraction preserve strict QoS while still delivering useful goodput under bounded execution windows, hard memory caps, and heterogeneous workload mixes? Implementation and evaluation therefore serve as the third and fourth objectives under controlled preemption pressure. Success at that stage means the runtime yields cleanly to resource revocation, remains stable under hard memory budgets, and demonstrates measurable useful goodput within bounded execution windows. The final written dissertation must then tie those results back to the argument of the proposal in a coherent architectural account.

2. State of the Art

The state of the art is best understood as four connected themes rather than as a paper-by-paper catalogue. Taken together, these themes define the opportunity, isolate the systems constraints, and motivate the precise gap addressed by this dissertation.

2.1. Theme 1: AI-RAN turns shared RAN infrastructure into a hosting substrate

At the architectural level, Open RAN and AI-RAN work establishes the conditions under which RAN infrastructure can become a shared substrate for AI services. Disaggregated distributed-unit and central-unit software (DU/CU), O-Cloud resources — the shared cloud-computing substrate attached to the RAN — and service-management-and-orchestration (SMO) control collectively move the RAN away from monolithic telecom appliances and toward a cloud-native control model [3, 1, 2]. Because that shift exposes programmable and accelerated infrastructure at the edge, it makes AI services viable as native tenants of the same physical system.

At the same time, however, the same literature makes clear that timing remains the governing obligation. CHRONOS shows that jitter, interruption, and network delay already consume a substantial fraction of slot-level budgets, that is, sub-millisecond deadlines tied to radio transmission slots [8]. Unlike data-centre environments where accelerators are dedicated, shared RAN compute must therefore be designed under hard timing constraints from the start. Many sharing proposals fail because they ignore that timing dominance: any AI-on-shared-RAN design must treat strict QoS preservation as a primary requirement rather than as a tunable optimisation target.

2.2. Theme 2: Latency-critical inference is heterogeneous and phase-sensitive

At the workload level, CORA is especially valuable because it models inference as a coupled uplink, compute, and downlink pipeline rather than as a GPU-only task [4]. That perspective matters because compatibility depends not only on accelerator pressure, but also on the radio path, namely the uplink and downlink transport path that carries inputs to the RAN edge and returns outputs to the device. Recent device–RAN–cloud studies reinforce that point by showing that model variant, service-level objective, and stage coupling jointly determine whether edge placement remains worthwhile under tight RAN-edge budgets [5].

To reason about that heterogeneity systematically, the project treats workload families as joint application/model classes with recurring execution structure. Stateless vision inference families, for example, often rely on convolutional or hybrid convolutional-transformer models for segmentation, pose estimation, or super-resolution; operationally, they consume a bounded input, execute one forward pass, and return one result. Streaming language families, by contrast, often rely on auto-regressive or encoder–decoder transformer models; operationally, they are phase-split into prompt prefill and token-by-token decoding, while preserving large key-value state across successive decode steps [6, 7, 9]. Automatic Speech Recognition (ASR) introduces another streaming regime in which audio arrives incrementally, partial hypotheses must be updated over time, and safe interruption boundaries depend on how much acoustic and linguistic context has already been accumulated. Rendering-oriented families introduce yet another regime in which useful goodput is measured in ray batches or rendering tiles rather than one completed request.

The central implication is that compatibility with shared RAN infrastructure depends on (i) execution boundaries, (ii) persistent-state semantics, and (iii) the extent to which accelerator occupancy can be bounded and reclaimed at runtime. In this proposal, bounded accelerator control means placing explicit caps on accelerator width, memory residency, and uninterrupted execution time so that preemption can occur before radio deadlines are endangered. What fails under RAN-first control is not inference in general, but the assumption that all inference workloads expose the same interruption boundary, state behaviour, and scheduler-visible unit of progress.

2.3. Theme 3: The relevant control mechanisms already exist, but not as one runtime

At the systems level, the literature already provides many of the mechanisms this dissertation needs, but those mechanisms appear as local solutions rather than as one integrated runtime for bounded RAN-edge execution. For stateless vision inference or pipeline-style services, systems such as *USHER*, *Orion*, *PPipe*, and *OctopInf* show that interference-aware consolidation, operator scheduling, portion planning, and memory-feasibility checks can determine when a request should be admitted, how much accelerator width it can safely consume, and whether concurrent execution remains stable [10, 11, 12, 13]. For streaming services, *PagedAttention*/vLLM, *DistServe*, *Orca*, *CacheGen*, and *DeepSpeed-Inference* show how key–value paging, iteration-level scheduling, phase disaggregation, context offloading, and parameter offload can preserve responsiveness when progress is stateful and iterative [6, 7, 9, 14, 15].

To address the challenge of revocable edge execution, these mechanisms are directly relevant to the proposed runtime. Key–value paging and context offloading become ways to preserve resumable progress under preemption. Memory-feasibility checks become admission constraints under bounded lease budgets. Fine-grained scheduling becomes the control lever that determines whether a workload can continue under a fixed execution window. The difficulty is therefore not the absence of mechanisms, but the absence of a unifying runtime contract that composes

those mechanisms under revocable RAN-edge conditions.

Fragmentation means that these systems generally assume stable data-centre GPU ownership, whereas the present setting must tolerate sudden millisecond-level, RAN-first resource revocation. The literature therefore supplies the pieces, but not the integrated control plane required when transient resources must remain subordinate to radio timing at every moment.

2.4. Theme 4: Empirical RAN studies and edge systems still stop short of inference-specific control

While Paradrop achieves lightweight edge deployment through explicit CPU, memory, network, and storage quotas together with practical lifecycle automation, it does not solve bounded accelerator control for heterogeneous inference [16]. While empirical studies of RAN deployments demonstrate that spare Streaming Multiprocessor (SM) headroom can emerge in bursty envelopes for secondary AI work, the literature still does not provide an inference-runtime contract for revocable shared GPU use. While CORA achieves a compelling end-to-end latency-budgeting model spanning uplink, compute, and downlink for edge inference, it does not specify the runtime-control layer that consumes those budgets under multi-tenant revocation pressure [4]. Consequently, the literature supplies the surrounding ingredients, but not the inference-specific runtime required for shared RAN infrastructure.

2.5. Gap statement

What remains missing is an inference-specific runtime-control architecture for shared RAN infrastructure. More precisely, the literature does not yet offer a unified design that combines (i) workload-family compatibility analysis, (ii) bounded execution windows, (iii) preemption-safe execution through context offloading, (iv) hard memory budgeting, (v) multi-tenant lifecycle control, and (vi) end-to-end latency awareness under a strict RAN-first operating model [4, 5, 16]. That integrated runtime gap is the one this dissertation addresses.

3. Implementation

3.1. Methodology or Approach

The project is organised as a phased systems study that maps directly to the three research questions.

Profiling establishes the workload baseline required for **RQ1**. Here, workload families are treated as recurrent execution classes of the kind distinguished in CORA-style categorisations: for example, stateless vision inference families such as image segmentation, pose estimation, super-resolution, or volume rendering; streaming language families such as auto-regressive language processing, sequence-to-sequence translation, and ASR pipelines [4, 5]. The profiling output is a structured matrix that records, for each representative family, the natural execution unit, dominant uplink/compute/downlink pressure, persistent-state profile, memory behaviour, safest interruption boundary, exact execution time per inference unit, Streaming Multiprocessor (SM) utilization, and radio resource blocks (PRBs) required for uplink and downlink streaming. It also profiles how those workloads physically perturb RAN-facing parameters, including slot-level deadline hit rates, PRB contention, and related control-channel pressure, so that later runtime decisions are grounded in measurable interaction with the primary telecom workload. This phase therefore establishes the evidential baseline on which later control decisions are justified.

Architecturally, the second phase converts that baseline into a runtime architecture that answers **RQ2**. The working design term is **ComputeLease**, used here to formalise bounded execution windows with duration, effective accelerator width, available memory headroom, radio resource blocks (PRBs) for uplink and downlink streaming, and revocation semantics [4, 5]. Around that interface, the project specifies the shared runtime primitives required across workload families: admission control; segmentation into preemption-safe execution units; context offloading for resumable execution; memory budgeting; tiered resource isolation, that is, progressively stricter separation among competing workloads as contention rises; and telemetry for control decisions. Unlike a new radio scheduler, this layer remains strictly inference-side. At the systems level, rather than treating inference as a GPU-only task, the runtime explicitly orchestrates the mandatory radio uplink for input data, the accelerator compute phase, and the downlink for results so that network bottlenecks and accelerator scheduling decisions are managed together.

Implementation then provides the executable artefact needed for **RQ3** through a containerised prototype in two steps. It first runs in an emulated RAN-edge interface built from a lightweight Kubernetes-based testbed with a simple synthetic RAN layer that replays trace-like envelopes to simulate bounded radio-resource dynamics, preemption events, and timing pressure in a controlled setting. That step is intended to show that the runtime architecture responds correctly to RAN-facing constraints before any deployment on a richer external platform. It then moves to brief integration on the Campus5G environment, treated here as the later edge-oriented Open RAN target rather than the first implementation step [17]. Across both steps, the prototype carries the shared runtime-control path plus a deliberately small set of workload adapters chosen to span heterogeneous execution styles, for example an object-detection service as stateless vision inference and a stateful streaming LLM decoder as a revocation-sensitive language workload. Operationally, a quota-aware lifecycle substrate inspired by Paradrop manages the slow path around the accelerator runtime so that non-GPU budgets, start-up cost, and auxiliary services are handled explicitly rather than implicitly [16].

Evaluation finally makes that prototype observable under revocation pressure through controlled experiments with bounded execution windows. Experiments will use replayed execution windows, synthetic resource variation, preemption events, hard memory caps, and heterogeneous workload mixes. The evaluation is organised around three linked questions: (i) whether the system preserves strict QoS by yielding cleanly to preemption without radio-stack redesign; (ii) whether it remains stable under coupled memory and revocation pressure, particularly for stateful workloads; and (iii) whether it converts bounded execution windows into sufficient useful goodput to justify the added control complexity.

3.2. Expected Outcomes

At the systems level, the project produces (i) a workload-property baseline that makes compatibility claims explicit, (ii) a runtime architecture specification that defines the shared control path, (iii) a containerised prototype that exercises the architecture under bounded execution, and (iv) an evaluation bundle containing repeatable experiments, plots, and analysis notes.

Scientifically, the project contributes a coherent account of how bounded, revocable RAN-edge execution can be converted into a useful substrate for edge inference without weakening strict QoS preservation. The goal is not to claim production readiness. Instead, the project demonstrates that the unified-runtime design is technically defensible, operationally bounded, and evaluable at dissertation scale.

3.3. Evaluation Criteria

Accordingly, the thresholds below are operational targets for dissertation success rather than claimed outcomes. For telecom-safe execution, the prototype must exhibit zero intentional cases in which inference continues after a preemption signal or lease expiry in the controlled experiments. To show that primary radio functions are not degraded, the evaluation tracks RAN workload parameters, including PRB utilization interference, control-channel degradation, and slot-level deadline violations for the primary telecom workloads. For inference service quality, the evaluation reports end-to-end latency, tail quantiles (p50/p95/p99), and deadline hit-rate for each workload family. For streaming language-model workloads, it additionally reports time-to-first-token (TTFT) and token throughput, because elevated TTFT is a useful proxy for queueing and prefill stalls under strict budgets [5]. Where language-model workloads are exercised, the experiments use explicit strict and medium latency targets aligned with that study and report how model variant or quantization changes feasibility under bounded resources.

For model fidelity, the evaluation also tracks workload-appropriate quality measures under tight resource constraints — for example, task accuracy for vision workloads and answer-quality or token-level correctness proxies for generative workloads — and reports any degradation relative to isolated baselines alongside the resource setting that caused it. For memory control, live memory must remain within configured budgets and context restoration must succeed for the workloads that rely on context offloading. For execution efficiency, the system must demonstrate that a substantial proportion of each bounded execution window is converted into useful goodput rather than scheduling overhead alone. The evaluation criteria therefore make the success conditions measurable while still leaving room for workload-specific analysis.

These same criteria structure the answers to the research questions. **RQ1** is answered if the workload matrix clearly separates compatible and incompatible workload families. **RQ2** is answered if the runtime-control design can be justified from those workload properties and mapped to implementable shared primitives. **RQ3** is answered if the prototype and experiments show bounded, preemption-safe execution under the target execution model.

3.4. Risk Assessment and Mitigations

The project risk profile is manageable because the design scope is bounded to the inference-runtime layer rather than a full RAN-stack redesign. The primary risks are therefore methodological and engineering risks rather than deployment or regulatory risks. Accordingly, the mitigation strategy is staged and evidence-led: produce design artefacts early, freeze the workload baseline before implementation, and preserve contingency for integration and writing late in the schedule.

Table 1: Risk profile and mitigation matrix.

Activity	Risk Description	Affects	Likelihood (H/M/L)	Severity (H/M/L)	Mitigation	Overall Category
Workload baseline	Workload families prove too broad for dissertation-scale coverage.	Scope and analysis depth	Medium	Medium	Freeze representative workloads in WP1; prefer depth over breadth; document exclusions explicitly.	Medium
Runtime design	The unified control abstraction becomes too ambitious or underspecified.	Architecture quality	Medium	High	Keep the design strictly at the inference-runtime boundary. Mitigate implementation overhead by refactoring and extending existing, extensible inference frameworks rather than engineering a new serving engine from scratch.	Medium-High
Prototype integration	Adapter and telemetry work take longer than expected.	Prototype completion	Medium	High	Implement the shared runtime skeleton first. Adopt an iterative approach by initially building adapters for simpler, easily integrated workload classes (e.g., stateless vision inference) before targeting complex stateful systems. Reserve a buffer in Weeks 11–12.	Medium-High
Evaluation	Controlled experiments do not produce sufficiently stable or interpretable results.	Evidence quality	Medium	Medium	Use replayable workloads, isolated baselines, fixed metrics, and incremental experiment scripts.	Medium
Facilities	Access to suitable GPU hardware is limited or intermittent.	Prototype and evaluation	Low	Medium	Use a portable container stack; emulate bounded execution windows if DU-adjacent hardware is unavailable.	Low-Medium
Writing schedule	Implementation overruns reduce dissertation writing time.	Submission quality	Medium	High	Start writing from Week 9, maintain chapter skeletons earlier, and protect a dedicated revision buffer.	Medium-High
Responsible research	Logging or traces inadvertently include sensitive operational detail.	Ethics and data handling	Low	Medium	Use synthetic or sanitised traces, minimise retention, and exclude subscriber or operator-identifiable data.	Low

3.5. Responsible Research Including Ethics

The project is a systems dissertation centred on controlled experimental infrastructure rather than live service deployment. Its main institutional risk is accidental misuse of traces, logs, or configuration artefacts, which will be controlled by using synthetic, replayed, or explicitly sanitised inputs and by retaining only the minimum material needed to explain and reproduce the experiments.

Some evaluated workloads may involve generative models or large language models (LLMs) at the edge. The relevant risk is unsafe or misleading model behaviour even when the runtime behaves correctly, so such workloads will be restricted to research-grade evaluation settings with bounded generation lengths, task-appropriate prompt restrictions, and lightweight output-side safety checks or refusal logging where available.

The project will not publish sensitive operator configuration, credentials, or deployment-ready hostile-use instructions. Third-party code and ideas will be cited explicitly, licence conditions will be checked before reuse, and experiment scripts, container configurations, and analysis notebooks will be kept under version control to preserve reproducibility. No personal data is expected in scope.

4. Research Plan, Milestones, and Deliverables

This section does not count towards the page limit.

Overall, the work is structured as four work packages so that the project plan matches the research logic. Each work package produces a concrete artefact, depends on the outputs of earlier stages, and contributes directly to the final dissertation.

4.1. Work packages

Table 2: Work-package structure and dependencies.

WP	Focus	Tasks	Depends on	Deliverable / milestone
WP1	Baseline and scope control	Consolidate literature; derive workload-property matrix; identify execution units, state, and interruption boundaries; freeze evaluation scope.	None	D1: workload baseline; M1: scope frozen
WP2	Unified runtime design	Specify <code>ComputeLease</code> ; define admission, segmentation, context offloading, tiered resource isolation, and memory-budget primitives; map mechanisms to workload families.	WP1	D2: architecture specification; M2: control design fixed
WP3	Prototype and adapters	Implement shared runtime skeleton; add workload adapters; add reclaim handling and telemetry; exercise the design in bounded execution experiments.	WP2	D3: runnable prototype; M3: integration complete
WP4	Evaluation, writing, and contingency	Run experiments; analyse results; write dissertation chapters; reserve buffer for integration slippage and revision.	WP1–WP3	D4: evaluation bundle; D5: dissertation draft/final; M4: submission-ready package

4.2. Resources, facilities, and dependencies

The project depends on access to a GPU-capable development environment, container tooling, and a reproducible experiment workflow. It also depends on access to the Campus5G network at the University of Edinburgh. It does not depend on live subscriber data. If DU-adjacent hardware is unavailable at a given stage, the dissertation remains valid because bounded execution can be studied through controlled replay and emulation before richer deployment assumptions are introduced. The project also depends on the availability of cited public systems evidence and the workload baseline derived from that evidence. These dependencies are manageable and do not require any change in scientific direction.

4.3. Deliverables and milestones

Table 3: Deliverables, timing, and dissertation use.

ID	Deliverable	Timing	Use in dissertation
D1	Workload-property matrix and compatibility baseline	Weeks 1–2	Grounds RQ1 and narrows evaluation scope
D2	Unified runtime-control specification	Weeks 3–5	Answers the design component of RQ2
D3	Containerised prototype with workload adapters	Weeks 6–8	Enables controlled experiments for RQ3
D4	Evaluation bundle, plots, and analysis notes	Weeks 9–10	Supports claims about safety, stability, and useful goodput
D5	Full dissertation draft, revisions, and final submission package	Weeks 9–12	Final integrated research output

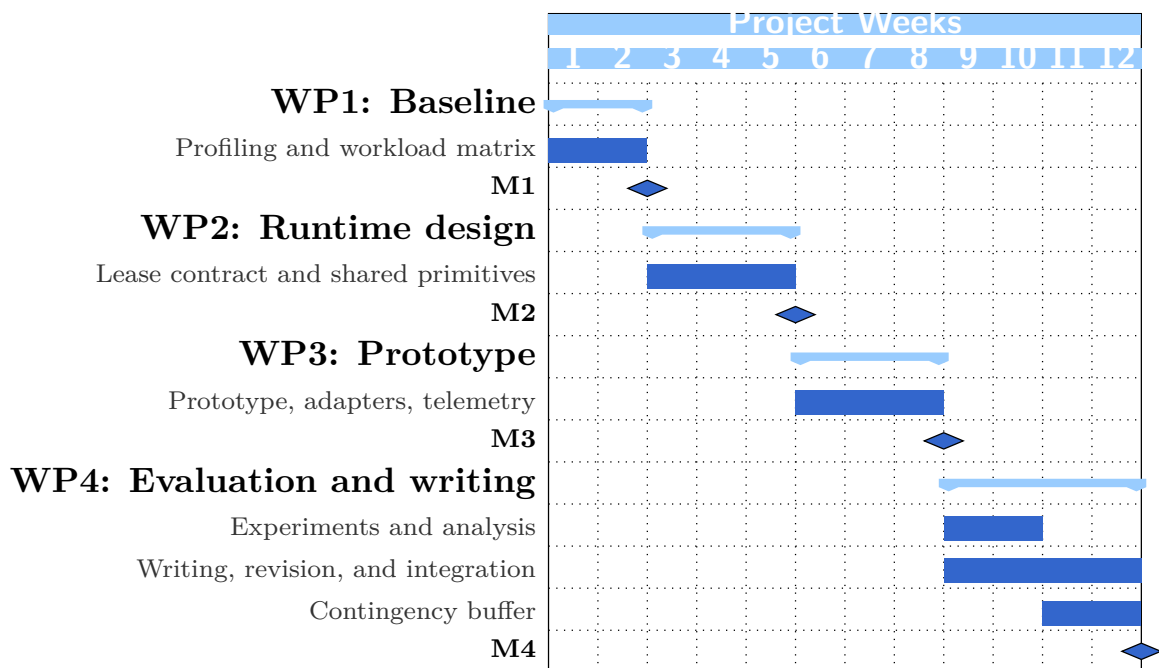


Figure 1: Twelve-week dissertation plan aligned with the work-package structure.

References

- [1] Michele Polese, Niloofar Mohamadi, Salvatore D'Oro, Leonardo Bonati, and Tommaso Melodia. Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G. *arXiv preprint arXiv:2507.06911*, 2025. doi:10.48550/arXiv.2507.06911.
- [2] Chenyuan Feng, Howard H. Yang, Kun Guo, Wenchao Xia, Chenxi Liu, and Tony Q. S. Quek. AI-RAN: The pathway to future wireless networks. *Journal of Information and Intelligence*, 4(1):5–22, 2026. doi:10.1016/j.jiixd.2025.12.013.
- [3] ACM. *Proceedings of the 2025 2nd ACM Workshop on Open and AI RAN*, Hong Kong, China, 2025. Association for Computing Machinery. Part of MobiCom '25. doi:10.1145/3737900.
- [4] Sunghyun Jin, Serae Kim, Sangtae Ha, and Kyunghan Lee. End-to-End Coordination of RAN and Edge Server for Latency-Critical Inference Serving over Cellular Networks. *Proceedings of the ACM on Networking*, 3(CoNEXT4):1–23, 2025. Article 40. doi:10.1145/3768987.
- [5] Hariz Yet, Nguyen Thanh Tam, Mao V. Ngo, Lim Yi Shen, Lin Wei, Jihong Park, Binbin Chen, and Tony Q. S. Quek. SLA-Aware Distributed LLM Inference Across Device-RAN-Cloud. In *IEEE INFOCOM Workshops 2026 (6G AI-RAN 2026)*, 2026. Preprint on arXiv:2602.23722. doi:10.48550/arXiv.2602.23722.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient Memory Management for Large Language Model Serving with PagedAttention. *arXiv preprint arXiv:2309.06180*, 2023. SOSP 2023. doi:10.48550/arXiv.2309.06180.
- [7] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, Santa Clara, CA, 2024. USENIX Association. URL: <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.
- [8] Ujjwal Pawar, Andrew E. Ferguson, Yuto Takano, Jon Larrea, Xenofon Foukas, Mahesh K. Marina, and Bozidar Radunovic. Towards Scalable and Cost-Effective RAN Emulation Leveraging the Public Cloud. In *Proceedings of the 26th International Workshop on Mobile Computing Systems and Applications*, HotMobile '25, pages 43–48, 2025. doi:10.1145/3708468.3711895.
- [9] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*. USENIX Association, 2022. URL: <https://www.usenix.org/system/files/osdi22-yu.pdf>.
- [10] Sudipta Saha Shubha, Haiying Shen, and Anand Iyer. USHER: Holistic Interference Avoidance for Resource Optimized ML Inference. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, 2024. URL: <https://www.usenix.org/system/files/osdi24-shubha.pdf>.
- [11] Foteini Strati, Xianzhe Ma, and Ana Klimovic. Orion: Interference-aware, Fine-grained GPU Sharing for ML Applications. In *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*, 2024. doi:10.1145/3627703.3629578.
- [12] Z. Jonny Kong, Qiang Xu, and Y. Charlie Hu. PPipe: Efficient Video Analytics Serving on Heterogeneous GPU Clusters via Pool-Based Pipeline Parallelism. In *2025 USENIX Annual Technical Conference (ATC 25)*. USENIX Association, 2025. URL: <https://www.usenix.org/system/files/atc25-kong.pdf>.
- [13] Thanh-Tung Nguyen, Lucas Liebe, Nhat-Quang Tau, Yuheng Wu, Jinghan Cheng, and Dongman Lee. OCTOPINF: Workload-Aware Inference Serving for Edge Video Analytics. *arXiv preprint arXiv:2502.01277*, 2025. doi:10.48550/arXiv.2502.01277.
- [14] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *ACM SIGCOMM 2024 Conference*, 2024. doi:10.1145/3651890.3672274.

- [15] Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Chao Li, Dacheng Li, Erwin Zheng, Olatunji Ruwase, Shaden Smith, Mengzhou Zhang, Jeff Rasley, and Yuxiong He. DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, 2022. doi:10.1109/SC41404.2022.00051.
- [16] Peng Liu, Dale Willis, and Suman Banerjee. Paradrop: Enabling Lightweight Multi-tenancy at the Network’s Extreme Edge. In *IEEE/ACM Symposium on Edge Computing (SEC)*, pages 1–13, 2016. doi:10.1109/SEC.2016.39.
- [17] Campus5G: A Campus Scale Private 5G Open RAN Testbed. Local paper in research paper/Campus5G.pdf. doi:10.1145/3787927.3787930.